

Findora: Conversational Recommender System

A multi-turn LLM **shop assistant** for smartphones & headphones. You chat with it in natural language ("a cheap phone with a good camera", "show me 2 cheaper ones", "why this one?", "compare those") and it recommends from a 1,000-product catalog (500 phones + 500 headphones).

It began as a structured LangGraph pipeline and was **re-architected into a hybrid tool-calling agent**: the LLM *plans* and *calls deterministic tools*, while the tools own all correctness (real catalog data, validated filters, honoured counts). The agent is the **default engine**; the pipeline is the fallback.

Quick start

Prerequisites: Python 3.10+ and one LLM API key (Gemini, OpenRouter, OpenAI, or Anthropic, see config below).

```
# 1. Create + activate a virtual environment
python -m venv .venv
source .venv/bin/activate          # macOS / Linux
# .venv\Scripts\activate          # Windows (PowerShell/cmd)

# 2. Install dependencies
pip install -r requirements.txt

# 3. Configure your API key
# open .env and paste your key/s (see "Configuration" below)

# 4. Run the app (from the project root)
streamlit run src/app.py
```

This opens the chat UI in your browser (default <http://localhost:8501>). No other setup needed — the product data ships in [datasets/](#), and the semantic model downloads itself on first run (~30 MB).

Configuration

All configuration is in a single file, **.env**. The only thing you *must* set is one LLM key:

- **Gemini** (default)
- **OpenRouter** (easiest — one key, many models): set **OPENROUTER_API_KEY**.
- **OpenAI**: set **LLM_PROVIDER=openai** and **OPENAI_API_KEY**.
- **Anthropic**: set **LLM_PROVIDER=anthropic** and **ANTHROPIC_API_KEY**.
- **Local (Ollama)**: set **LLM_PROVIDER=ollama** (and run Ollama). You can also switch models live from the in-app sidebar picker.

Optional: inside **.env**: **ENGINE=pipeline** runs the legacy v1 engine; **LANGCHAIN_API_KEY** enables LangSmith tracing. Every key is documented inline in **.env.example**.

Running things

```
streamlit run src/app.py           # the chat app (agent engine, default)
python src/test_cli.py            # quick CLI chat (no browser; good for a fast
check)
python src/test_tools.py          # fast deterministic tool unit-tests (no API key
needed)
python src/eval_harness.py        # full A/B evaluation -> eval_agent.json (uses
the LLM; a few minutes)
```

All commands are run from the **project root** (the code lives in `src/`, the data in `datasets/`).

To run the **legacy pipeline** instead of the agent (for the app or the eval), set `ENGINE=pipeline` in your `.env`. `eval_harness.py` then writes `eval_baseline.json`.

How it works (in brief)

```

your message
|
▼ LLM plans → calls deterministic tools → observes results
  (system policy + 6 tool schemas)      (search · details · compare ·
                                         explain · top-picks · catalog)
  ▲                                     ─── loop up to 4 rounds ───
|
▼
grounded reply + product cards (action: recommend | compare | respond)
```

- **The LLM decides *what to do*; deterministic code decides *what is true*.** The tools return only real catalog rows, validate every filter, honour an explicit count, and report out-of-catalog names honestly, so the model never invents a product or a spec.
- **Ranking is TOPSIS** (a transparent multi-criteria score, Hwang & Yoon 1981) by default, with a numpy logistic learn-to-rank model that takes over once enough 👍 click-feedback exists.
- **Semantic "vibe" search** (`model2vec` static embeddings, no GPU) handles requests like "good for travel" that structured filters can't express; falls back to numpy TF-IDF offline.
- **Deterministic guardrails** handle the must-be-exact bits (undo = "ignore that"; brand exclusion).

Data & enrichment

- `datasets/reduced_file_smartphone_500.csv`, `datasets/reduced_file_headphones_500.csv` — the catalog (structured specs; price in USD).
- `datasets/image_cache.json`, offline product images (joined at load time; cards render them).
- `datasets/smartphone_enrichment.json`, a grounded **description** and a clearly-labeled **AI review summary** for every phone, generated from its real specs by `enrich_smartphones.py`. These show in a product's **Details** view and feed the semantic search. (Generating real third-party reviews was attempted but not viable for this catalog; see `CHANGES.md`.)

Project structure

```

├─ src/                                # all application code (run commands from the
project root)
|   └─ app.py                          # Streamlit chat UI ← main entry point (streamlit
run src/app.py)
|   └─ agent.py                        # v2 tool-calling agent: plan→call→observe loop +
policy + guardrails
|   └─ tools.py                        # the 6 validated tools + OpenAI function-calling
schemas
|   └─ graph.py                        # run_turn() dispatch: ENGINE=agent (default) |
pipeline
|   └─ nodes.py, nlu.py, schema.py, resolver.py  # v1 pipeline + NLU stack (the
deterministic core)
|   └─ database.py                    # CSV loader + structured filters + TOPSIS scoring
+ enrichment join
|   └─ semantic.py                    # dense-embedding / TF-IDF "vibe" retrieval
|   └─ ranking.py                     # pluggable ranker (TOPSIS ↔ learned) + click-
feedback flywheel
|   └─ memory.py, observability.py, llm_client.py, state.py
|   └─ enrich_smartphones.py          # offline: generate grounded descriptions + AI
review summaries
|   └─ enrich_images.py, images.py      # offline: product-image cache
|   └─ eval_harness.py                 # persona sim + LLM-judge + 28 adversarial/safety
checks (the A/B gate)
|   └─ make_poster.py                  # generates poster.pdf (run: python
src/make_poster.py)
|   └─ test_tools.py, evaluate.py, test_cli.py # tests / deterministic suites
├─ datasets/                          # product CSVs + image cache + enrichment
├─ poster.pdf                          # the final poster
├─ CHANGES.md
└─ requirements.txt, .env.example

```

Notes: all code lives in `src/`; run every command from the **project root** (e.g. `streamlit run src/app.py`, `python src/eval_harness.py`) so the relative `datasets/` path resolves. `logs/`, `profiles/`, and `models/` are created automatically at runtime. Do **not** commit/zip `.venv/` or `.env`.